ADVANCED AUTOMATION ENGINEERING PROGRAM

Synchronization

Auto-waiting · explicit waits · dynamic elements · flake-free tests

SPRINT 1

DAY 3 of 42

Jai Singh

Week 1 — UI Automation (Playwright) · Full-day session (3h teach + 5h hands-on)

Planner ref: Week 1 · Day 3 — Synchronization

01 How Today Runs

SESSION MAP

Day 2 automated a login. But real apps are slow and asynchronous — buttons appear late, data loads over the network. Today is about making tests wait correctly, so they stop being flaky.

09:30	**Concepts**	Why tests flake, auto-waiting, actionability
11:00	**Waits, live**	Explicit waits, dynamic elements, network — in the IDE
12:00	**Guided exercises**	Short 'you try' tasks with checkpoints
13:30	**Lab (staged)**	Stabilise a flaky login in gated milestones
16:30	**Debrief & recap**	Flake patterns, evaluation, Day-4 preview

02 What We Cover Today

Today is about timing: making the test wait for the app to be ready, the right way — never with fixed sleeps.

Why tests flake
The root cause of timing bugs

Auto-waiting
What Playwright does for you

Explicit waits
When and how to wait yourself

Dynamic elements
Late-appearing UI

Network delays
Wait for responses

Flake-free tests
Stable, repeatable runs

Aligned to Planner **Wk1/Day3 “Synchronization”. ToC today:** auto-waiting, explicit waits, dynamic elements, network delays, avoiding flaky tests. **Deferred:** frames/popups→D4, traces/video→D5.

03 Learning Outcomes

By the end of today you will be able to:

- Explain why timing causes flaky tests, and how auto-waiting prevents most of it.
- Describe Playwright's actionability checks and web-first assertion retries.
- Use explicit waits (waitFor) for visibility, hidden, and attached states.
- Wait for a network response when the UI depends on data loading.
- Handle dynamic, late-appearing elements without fixed sleeps.
- Recognise and remove the #1 anti-pattern: hard-coded waits / sleeps.

04 Why Tests Flake

CONCEPT

A flaky test passes sometimes and fails other times with no code change. The usual cause is timing — the test acts before the app is ready.

The app is asynchronous

Data loads over the network, animations run, elements render late. The page is not ready the instant it appears.

The test is too fast

In weaker models, code clicks before the UI is ready. Playwright auto-waiting prevents much of this — but raw reads and premature assertions can still flake.

From Selenium: this is the classic 'element not interactable' / StaleElementReferenceException — usually patched with Thread.sleep(), which we will not do here.

The fix is synchronization: make the test wait for the right condition, not a fixed amount of time.

05 Auto-Waiting — The Core Idea

CONCEPT

Before most actions, Playwright automatically waits for the element to be ready. You usually do not wait manually at all.

```
// Playwright waits for the button to be visible, enabled,
// stable and able to receive the click — then clicks. No sleep.
await page.getByRole('button', { name: 'Sign in' }).click();
```

Actions auto-wait

click, fill, check, selectOption and more wait for actionability first.

Assertions retry

web-first expect(...) polls until the condition is true or it times out.

06 What 'Ready' Means: Actionability

CONCEPT

Before an action like click(), Playwright waits for ALL of these to be true — automatically:

1	**Resolved in the DOM**	The locator resolves to an element that exists
2	**Visible**	It has a non-empty bounding box and is not hidden
3	**Stable**	It has stopped animating / moving
4	**Enabled**	It is not disabled
5	**Receives events**	It is not obscured by another element

If any check fails, Playwright keeps retrying until it passes or the timeout is reached.

07 Web-First Assertions Retry

CONCEPT

expect(locator) assertions are 'web-first': they automatically retry until the condition is met or the timeout expires. This is what lets you assert on things that have not loaded yet.

```
// retries until the heading appears (data still loading) — or times out
await expect(page.getByRole('heading', { name: /welcome/i }))
      .toBeVisible();

// retries until the cart count becomes '3'
await expect(page.getByTestId('cart-count')).toHaveText('3');
```

⚠ **Note:** only expect(locator) retries. A plain locator.textContent() reads once — wrap values you are waiting on in an expect.

08 The Anti-Pattern: Hard Sleeps

✗ Don't

- waitForTimeout(3000) before acting
- Too short → flaky; too long → slow
- Guesses time instead of a condition
- Selenium's Thread.sleep() equivalent

✓ Do

- Let actions auto-wait
- Use web-first expect(...) to retry
- Wait for a condition or state
- Wait for a network response if needed

```
await page.waitForTimeout(3000);  // ✗ avoid — use a condition, not a guess
```

09 When Auto-Wait Isn't Enough

CONCEPT

Auto-waiting covers most cases. You add an explicit wait only when you need to wait for something that is not tied to your next action.

Element appears late	A spinner finishes and the real content renders after a delay
Data loads over network	The UI updates only after an API response arrives
State changes without your action	A banner auto-dismisses, a toast disappears, a count updates

Even then, prefer a web-first assertion over a manual wait where you can.

10 Explicit Waits — waitFor States

HANDS-ON

locator.waitFor() waits for an element to reach a state. Useful when you must wait before doing something other than an action.

```
// wait until the spinner is gone before reading the page
await page.getByTestId('spinner').waitFor({ state: 'hidden' });

// wait until a late element is visible
await page.getByRole('heading', { name: /dashboard/i })
      .waitFor({ state: 'visible' });
```

States: `attached` (in DOM) · `visible` (shown) · `hidden` (gone/invisible) · `detached` (removed from DOM)

11 Coming From Java — Waits Mapped

BRIDGE

If you used Selenium's WebDriverWait in college, here is how the same ideas look in Playwright — with far less ceremony.

SELENIUM (JAVA)		PLAYWRIGHT
`Thread.sleep(3000)`	→	`avoid in tests — use condition waits`
`new WebDriverWait(driver, 10)`	→	`built-in auto-wait + timeouts`
`.until(visibilityOf(el))`	→	`expect(loc).toBeVisible()`
`.until(elementToBeClickable)`	→	`auto-waited by click()`
`.until(invisibilityOf(el))`	→	`loc.waitFor({ state: 'hidden' })`
`.until(textToBe(el, '3'))`	→	`expect(loc).toHaveText('3')`
`FluentWait + polling`	→	`built-in retry (no setup)`

Key shift: in Selenium you wire up waits explicitly; in Playwright they are built in — you mostly just assert.

12 Waiting for the Network

HANDS-ON

When the UI updates only after an API call, you can wait for that response explicitly — useful for data-driven screens.

```
// start waiting BEFORE the click that triggers the request
const respPromise = page.waitForResponse(resp =>
  resp.url().includes('/api/login') && resp.status() === 200);
await page.getByRole('button', { name: 'Sign in' }).click();
await respPromise;  // now the response has arrived

await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();
```

Set up the wait first, then act — otherwise the response can arrive before you start listening.

13 Handling a Delayed Login Button

HANDS-ON

IN THE IDE the demo app enables 'Sign in' after a short delay — let's make the test wait correctly.

```
test('waits for the delayed sign-in button', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@test.com');
  await page.getByLabel('Password').fill('Secret123');
  // no sleep — click() auto-waits until the button is enabled
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();
});
```

The point: click() already waits for 'enabled' — the delayed button needs no special handling.

14 Run It & Observe

HANDS-ON

IN THE IDE run headed and watch: the test pauses on the disabled button, then clicks the instant it enables.

```
npx playwright test sync.spec.ts --headed
```

Watch the auto-wait happen

```
npx playwright test sync.spec.ts
```

Run headless

```
npx playwright test sync.spec.ts --repeat-each=5
```

Re-run to check for flakiness

What to look for: the test does not fail on the disabled button, and passes consistently across repeated runs.

15 Timeouts & Configuration

CONCEPT

Waits do not run forever. Each has a timeout, configurable globally or per call. Tune them — do not disable them.

```
// playwright.config.ts — sensible global defaults
export default defineConfig({
  timeout: 30_000,          // per-test budget
  expect: { timeout: 5_000 }, // web-first assertion retry window
});

// per-call override when one element is genuinely slow
await expect(slow).toBeVisible({ timeout: 15_000 });
```

Prefer fixing the wait condition over inflating timeouts. Big timeouts hide real problems and slow the suite.

GUIDED PRACTICE

Guided Exercises

Three short 'you try' tasks on synchronization. Attempt each in the IDE, then we review together.

~30 minutes · individual · ask for help anytime

17 Exercise 1 — Remove the Sleep

YOU TRY

This test sleeps for 3 seconds hoping the welcome heading has appeared. Replace the guess with a web-first assertion that waits for the right condition.

Your task:

- Delete the waitForTimeout line.
- Assert the welcome heading is visible — let the assertion retry.

IN THE IDE edit the test — swap the sleep for an assertion.

```
await page.getByRole('button', { name: 'Sign in' }).click();
await page.waitForTimeout(3000);  // TODO: replace with an assertion
```

18 Exercise 1 — Solution

SOLUTION

```
await page.getByRole('button', { name: 'Sign in' }).click();
// retries until the heading shows — no fixed wait
await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();
```

Why: the assertion polls until the heading appears or the timeout hits. It is faster when the app is quick and reliable when it is slow — the fixed 3s was both flaky and wasteful.

From Selenium: this replaces both Thread.sleep() and an explicit WebDriverWait.until(visibilityOf(...)).

19 Exercise 2 — Wait for the Spinner

YOU TRY

After clicking 'Load', a spinner shows while data fetches, then the results table appears. Make the test wait for the spinner to disappear before checking the table.

Your task:

- Assert the spinner (testid 'spinner') becomes hidden.
- Then assert the results table is visible.

IN THE IDE assert the spinner is hidden, then assert the table is visible.

```
await page.getByRole('button', { name: 'Load' }).click();
// TODO: assert spinner hidden, then assert table visible
```

20 Exercise 2 — Solution

SOLUTION

```
await page.getByRole('button', { name: 'Load' }).click();
// assert the spinner is gone — retries automatically
await expect(page.getByTestId('spinner')).toBeHidden();
// now the data is in — assert the table
await expect(page.getByRole('table')).toBeVisible();
```

Why: web-first toBeHidden() / toBeVisible() retry until the condition holds — matching our 'assert the state' habit. Use waitFor({ state: 'hidden' }) instead only when you must wait before a non-assertion step.

21 Exercise 3 — Wait for the Response

YOU TRY

Clicking 'Sign in' triggers POST /api/login. The welcome screen only renders after a 200 response. Make the test wait for that response, then assert.

Your task:

- Set up waitForResponse for /api/login (status 200) BEFORE the click.
- Click, await the response, then assert the welcome heading.

IN THE IDE start the response wait first, then click — order matters.

```
// TODO: const respPromise = page.waitForResponse(...)
await page.getByRole('button', { name: 'Sign in' }).click();
// TODO: await respPromise; then assert welcome heading
```

22 Exercise 3 — Solution

SOLUTION

```
const respPromise = page.waitForResponse(r =>
  r.url().includes('/api/login') && r.status() === 200);
await page.getByRole('button', { name: 'Sign in' }).click();
await respPromise;
await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();
```

⚠ **Lesson:** create the response promise BEFORE the click — otherwise the response may arrive before you start listening.

PROJECT WORK

Afternoon Lab

Stabilise a flaky login flow yourself, in five staged milestones. Each has an acceptance gate before you move on.

~4 hours · work in the IDE · trainer circulates

24 Lab Overview

Goal: take a flaky login spec and make it pass reliably across repeated runs — using auto-wait, waitFor, and network waits, with zero hard sleeps.

1	**Reproduce the flakiness**	Run the given spec with --repeat-each; watch it fail intermittently
2	**Remove hard sleeps**	Delete waitForTimeout; rely on auto-wait + assertions
3	**Handle the delayed button**	Confirm click() waits for the enabled state
4	**Wait for the network**	Wait for /api/login before asserting the result
5	**Stabilise & re-run**	Pass 5x in a row; commit clean

25 Milestone 1 — Reproduce the Flakiness

LAB

Before fixing anything, see the problem. The provided spec reads data before the API-driven UI has loaded — run it several times and watch it flake.

IN THE IDE run the given sync.spec.ts repeatedly; note WHY each failure happens.

```
npx playwright test sync.spec.ts --repeat-each=10
# the spec does a raw read before the data is in:
const c = await page.getByTestId('cart-count').textContent();
expect(c).toBe('3');  // reads once — fails when data is slow
```

✓ **Gate:** you have seen an intermittent failure and can name the cause: a premature read before the API-driven UI loaded.

26 Milestone 2 — Remove Hard Sleeps

LAB

Strip out every waitForTimeout and fix the premature read from Milestone 1 — let assertions retry.

IN THE IDE delete waitForTimeout; replace the raw textContent read with a retrying assertion.

```
// ✗ before — reads once, flaky when data is slow
const c = await page.getByTestId('cart-count').textContent(); expect(c).toBe('3');
// ✓ after — retries until the count is '3'
await expect(page.getByTestId('cart-count')).toHaveText('3');
```

✓ **Gate:** no waitForTimeout remains in the spec; the test still passes.

27 Milestone 3 — Handle the Delayed Button

LAB

The 'Sign in' button enables after a delay. Confirm your test handles it with auto-wait alone — no extra code.

IN THE IDE fill the form and click; verify click() waits for the enabled state.

```
// click auto-waits for 'enabled' — no waitFor needed here
await page.getByRole('button', { name: 'Sign in' }).click();
await expect(page).toHaveURL(/\/dashboard/);
```

✓ **Gate:** the test clicks the button once it enables, with no explicit wait or sleep added.

28 Milestone 4 — Wait for the Network

LAB

The dashboard data loads via an API call. Wait for that response before asserting on the data.

IN THE IDE set up waitForResponse before the action that triggers it.

```
const resp = page.waitForResponse(r =>
  r.url().includes('/api/profile') && r.status() === 200);
await page.getByRole('link', { name: 'Profile' }).click();
await resp;
await expect(page.getByText('user@test.com')).toBeVisible();
```

✓ **Gate:** the data assertion runs only after a 200 response; passes without a sleep.

29 Milestone 5 — Stabilise & Re-run

LAB

Prove the test is stable: it should pass every time, not just once.

IN THE IDE run repeatedly; if any run flakes, find the remaining unconditioned wait.

```
npx playwright test sync.spec.ts --repeat-each=5
# expect: 5/5 passes, no flakes
```

✓ **Gate:** 5 consecutive green runs; no hard sleeps; branch committed clean.

Tip: --repeat-each is your flake detector. 5/5 green is a minimum stability signal for this lab — not proof a test can never flake.

30 Common Pitfalls

WATCH-OUTS

Hard sleeps
waitForTimeout 'fixes' flakiness by luck — it returns later.

Reading without retry
textContent() reads once; wrap waited values in expect().

Listening too late
Set up waitForResponse BEFORE the click that triggers it.

Inflating timeouts
Huge timeouts hide real bugs and slow the suite.

waitForSelector everywhere
Prefer assertions / auto-wait; reach for waitFor sparingly.

networkidle as a crutch
Fragile on busy apps; wait for the specific response instead.

31 Debrief — Is Your Test Stable?

Before you call a test stable, check it against these.

- **Zero hard sleeps?** No waitForTimeout anywhere in the spec.
- **Waits tied to conditions?** Every wait is for a state, response, or assertion — not a duration.
- **Listening before acting?** Network waits are set up before the triggering action.
- **Passes on repeat?** Green across --repeat-each, not just one lucky run.
- **Timeouts sane?** Tuned, not inflated to mask a real delay.

32 How Today Maps to Evaluation

ASSESSMENT

Day 3 contribution areas for the Week 1 mini-capstone. The full rubric is scored out of 100 at the weekly pair demo — here is where each area comes from today:

Concept understanding	Explain auto-wait, actionability, retries	15
Hands-on implementation	Stable login spec in Git	30
Synchronization quality	Correct waits; zero hard sleeps	15
Assertions & coverage	Web-first assertions for loaded state	15
Stability / flake-free	Passes across repeated runs	10
Git hygiene	Branch, commit, clean tree	10
Collaboration & demo	Explain the flake fix clearly	5
Total	Weekly mini-capstone score	**100**

33 Key Takeaways

- Flaky tests are usually timing bugs — the test acts before the app is ready.
- Playwright auto-waits for actionability before actions; web-first assertions retry.
- Reach for explicit waitFor (visible / hidden / attached / detached) only when needed.
- Wait for a specific network response when the UI depends on loaded data — listen first, then act.
- Never use hard sleeps (waitForTimeout) — wait for a condition, not a duration.
- Tune timeouts; verify stability with --repeat-each.

34 Quick Reference

Pin this — the waiting patterns you will reuse whenever a test goes flaky.

Prefer (auto / assert)

```
// just act — auto-waits
await loc.click();
// retry until true
await expect(loc)
  .toBeVisible();
await expect(loc)
  .toHaveText('3');
```

When you must wait

```
// element state
await loc.waitFor(
  { state: 'hidden' });
// network (listen first)
const r = page.waitForResponse(
  resp => resp.url()
    .includes('/api/x') &&
    resp.status() === 200);
await loc.click();
await r;
```

Day 3 complete

You can explain why tests flake and fix it the right way — auto-waiting, explicit waits, and network waits, with zero hard sleeps.

Next — Day 4: Navigation, Frames & Accessibility — multi-page flows, iframes, and automated a11y checks with Axe.